



Assignment Solutions : Greedy 1

Q1 There is a bag that consists of items, each item has a number 1, 0, or -1 written on it.

You are given four non-negative integers `numOnes`, `numZeros`, `numNegOnes`, and `k`.

The bag initially contains:

`numOnes` items with 1s written on them.

`numZeros` items with 0s written on them.

`numNegOnes` items with -1s written on them.

We want to pick exactly `k` items among the available items. Return the maximum possible sum of numbers written on the items.

Example 1:

Input: `numOnes = 3`, `numZeros = 2`, `numNegOnes = 0`, `k = 2`

Output: 2

Explanation: We have a bag of items with numbers written on them {1, 1, 1, 0, 0}. We take 2 items with 1 written on them and get a sum in a total of 2.

It can be proven that 2 is the maximum possible sum.

Example 2:

Input: `numOnes = 3`, `numZeros = 2`, `numNegOnes = 0`, `k = 4`

Output: 3

Explanation: We have a bag of items with numbers written on them {1, 1, 1, 0, 0}. We take 3 items with 1 written on them, and 1 item with 0 written on it, and get a sum in a total of 3.

It can be proven that 3 is the maximum possible sum.

Solution :

Code :

```
class Solution {
public:
    int kItemsWithMaximumSum(int numOnes, int numZeros, int
numNegOnes, int k) {
        int answer = 0;
        if(k < numOnes)return k;
        k -= numOnes;
        answer += numOnes;

        if(k < numZeros)return answer;
        k -= numZeros;

        if(k < numNegOnes)return answer - k;
        k -= numNegOnes;
        answer -= numNegOnes;

        return answer;
    }
};
```

Q2 Given an integer k, return the minimum number of Fibonacci numbers whose sum is equal to k. The same Fibonacci number can be used multiple times.

The Fibonacci numbers are defined as:

F1 = 1

F2 = 1

$F_n = F_{n-1} + F_{n-2}$ for $n > 2$.

It is guaranteed that for the given constraints we can always find such Fibonacci numbers that sum up to k.

Example 1:

Input: k = 7

Output: 2

Explanation: The Fibonacci numbers are: 1, 1, 2, 3, 5, 8, 13, ...

For k = 7 we can use 2 + 5 = 7.

Example 2:

Input: k = 10

Output: 2

Explanation: For k = 10 we can use 2 + 8 = 10.

Example 3:**Input: k = 19****Output: 3****Explanation: For k = 19 we can use $1 + 5 + 13 = 19$.****Solution :****Code :**

```
class Solution {
public:
    int findMinFibonacciNumbers(int k) {
        vector<long long> fib;
        fib.push_back(1);
        fib.push_back(1);

        // generate fibs less than equal to k
        for(int i=2;;i++)
        {
            if( (fib[i-1]+fib[i-2]) > k)
                break;
            fib.push_back(fib[i-1]+fib[i-2]);
        }
        int c = 0;
        int j = fib.size()-1;

        // select greedily and keep subtracting till 0
        while(true){
            long long x = fib[j];
            c += k/x;
            k%=x;
            if(k==0)
                break;
            j--;
        }
        return c;
    }
};
```

Q3 You are given a non-negative integer array nums. In one operation, you must:**Choose a positive integer x such that x is less than or equal to the smallest non-zero element in nums.****Subtract x from every positive element in nums.****Return the minimum number of operations to make every element in nums equal to 0.**

Example 1:

Input: nums = [1,5,0,3,5]

Output: 3

Explanation:

In the first operation, choose x = 1. Now, nums = [0,4,0,2,4].

In the second operation, choose x = 2. Now, nums = [0,2,0,0,2].

In the third operation, choose x = 2. Now, nums = [0,0,0,0,0].

Example 2:

Input: nums = [0]

Output: 0

Explanation: Each element in nums is already 0 so no operations are needed.

Solution :

Code :

```
class Solution {
public:
    int minimumOperations(vector<int>& a) {
        unordered_set<int>s;
        for(auto x:a){
            if(x != 0)s.insert(x);
        }

        return (int)s.size();
    }
};
```

Q4 You are given a positive integer n, you can do the following operation any number of times:

Add or subtract a power of 2 from n.

Return the minimum number of operations to make n equal to 0.

A number x is power of 2 if $x == 2^i$ where $i \geq 0$.

Example 1:

Input: n = 39

Output: 3

Explanation: We can do the following operations:

- Add $20 = 1$ to n , so now $n = 40$.
- Subtract $23 = 8$ from n , so now $n = 32$.
- Subtract $25 = 32$ from n , so now $n = 0$.

It can be shown that 3 is the minimum number of operations we need to make n equal to 0.

Example 2:

Input: $n = 54$

Output: 3

Explanation:

We can do the following operations:

- Add $21 = 2$ to n , so now $n = 56$.
- Add $23 = 8$ to n , so now $n = 64$.
- Subtract $26 = 64$ from n , so now $n = 0$.

So the minimum number of operations is 3.

Solution :

Code :

```
class Solution {
public:
    int minOperations(int n) {
        int x = 1, y = 1;
        if(n ≤ 0) return 0;
        if(n = 1) return 1;
        while(x * 2 < n) x *= 2;
        y = x * 2 - n;
        x = n - x;
        int low = minOperations(x);
        int high = minOperations(y);
        return min(low, high) + 1;
    }
};
```

Q5 Given two arrays $a[]$ and $b[]$ of the same size. Task is to find the minimum sum of two elements such that they belong to different arrays and are not at the same index in their arrays.

Example :

Input : a[] = {5, 4, 13, 2, 1}

b[] = {2, 3, 4, 6, 5}

Output : 3

We take 1 from a[] and 2 from b[]

Sum is $1 + 2 = 3$.

Input : a[] = {5, 4, 13, 1}

b[] = {3, 2, 6, 1}

Output : 3

We take 1 from a[] and 2 from b[].

**Note that we can't take 1 from b[]
as the elements can not be at same
index.**

Solution :

Code :

```
#include <bits/stdc++.h>
using namespace std;

int minSum(int a[], int b[], int n) {
    int minA = a[0], indexA;
    for (int i=1; i<n; i++){
        if (a[i] < minA){
            minA = a[i];
            indexA = i;
        }
    }

    int minB = b[0], indexB;
    for (int i=1; i<n; i++){
        if (b[i] < minB){
            minB = b[i];
            indexB = i;
        }
    }
    if (indexA != indexB)
        return (minA + minB);
    int minA2 = INT_MAX, indexA2;
    for (int i=0; i<n; i++){
        if (i != indexA && a[i] < minA2){
            minA2 = a[i];
            indexA2 = i;
        }
    }
```

```
}
int minB2 = INT_MAX, indexB2;
for (int i=0; i<n; i++){
    if (i != indexB && b[i] < minB2){
        minB2 = b[i];
        indexB2 = i;
    }
}
return min(minB + minA2, minA + minB2);
}
int main(){
    int a[] = {5, -4, 3, -8, 1};
    int b[] = {2, 3, 4, -2, 1};
    int n = 5;
    cout << minSum(a, b, n);
    return 0;
}
```

